

Course Machine Learning

Course #7 – KNN Classification

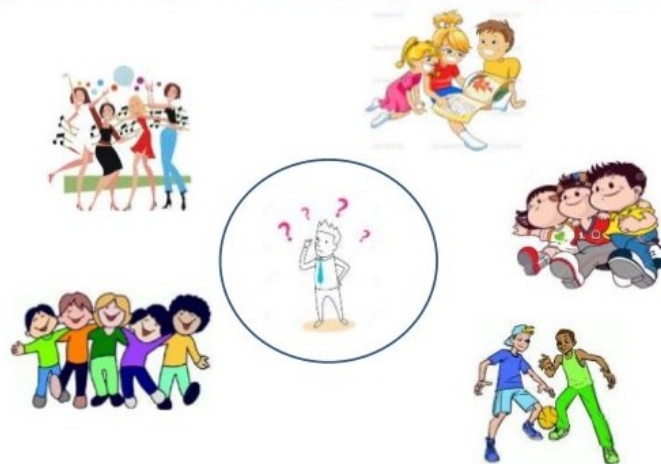
Professor
Abdelouahab Moussaoui



1 — K-Nearest Neighbors Classification

K-nearest neighbors (kNN) is a supervised machine learning algorithm that can be used to solve both classification and regression tasks. I see kNN as an algorithm that comes from *real life*. People tend to be effected by the people around them. Our behaviour is guided by the friends we grew up with. Our parents also shape our personality in some ways. If you grow up with people who love sports, it is higly likely that you will end up loving sports. There are ofcourse exceptions. kNN works similarly.

Tell me about your friends(who your neighbors are) and *I will tell you who you are.*



What is K-NN?

K-NN is a **non-parametric** and **lazy learning algorithm**. Non-parametric means there is no assumption for underlying data distribution i.e. the model structure is determined from the dataset.

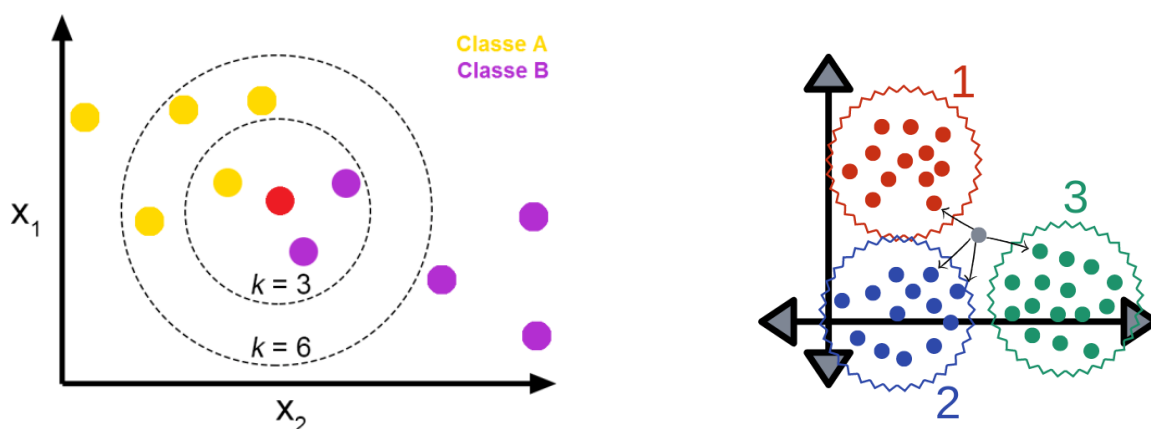
It is called **Lazy algorithm** because it **does not need any training data points for model generation**. All training data is used in the testing phase which makes training faster and testing phase slower and costlier.

K-Nearest Neighbor (K-NN) is a simple algorithm that stores all the available cases and **classifies the new data or case based on a similarity measure**.

K-NN classification

In *K-NN classification*, the output is a **class membership**. An object is classified by a **plurality vote of its neighbors**, with the object being assigned to the **class most common among its k nearest neighbors** (k is a positive integer, typically small). If $k = 1$, then the object is simply assigned to the class of that single nearest neighbor.

To determine which of the K instances in the training dataset are most similar to a new input, **a distance measure is used**. For real-valued input variables, the most popular distance measure is the **Euclidean distance**.



The Red point (in the left image) is classified to the class most common among its k nearest neighbors.

The Euclidean distance

- **The Euclidean distance** — is the most common distance metric used in **low dimensional data sets**. It is also known as the **L2 norm**. The Euclidean distance is the usual manner in which distance is measured in the real world.

where $\mathbf{p}$ and $\mathbf{q}$ are n -dimensional vectors and denoted by $\mathbf{p} = (p_1, p_2, \dots, p_n)$ and $\mathbf{q} = (q_1, q_2, \dots, q_n)$ represent the n attribute values of two records.

- While Euclidean distance is useful in low dimensions, it **doesn't work well in high dimensions and for categorical variables**. The drawback of Euclidean distance

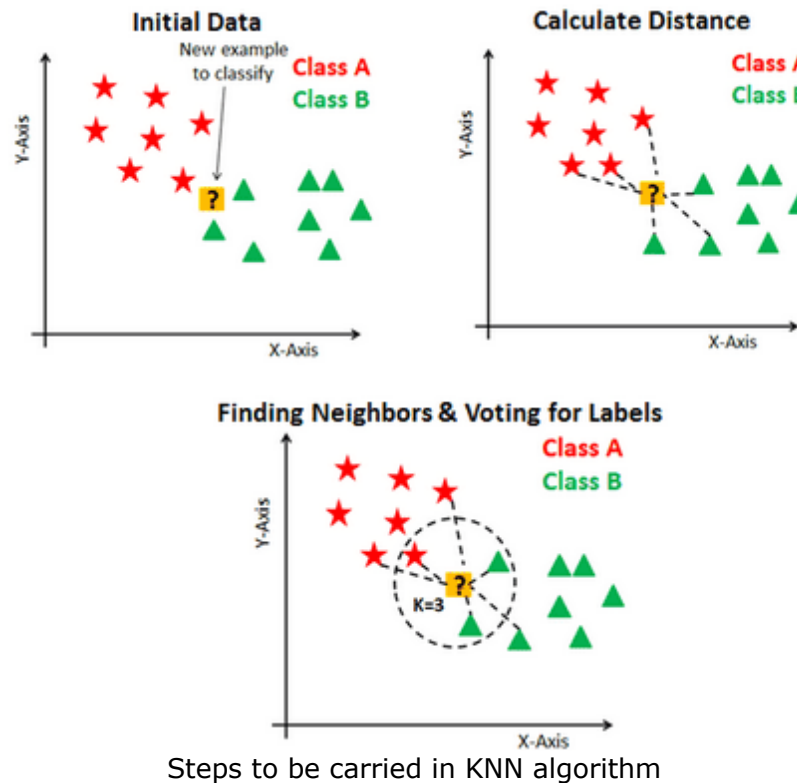
is that it **ignores the similarity between attributes**. Each attribute is treated as totally different from all of the attributes.

Other popular distance measures:

- **Hamming Distance** — Calculate the distance between binary vectors.
- **Manhattan Distance** — Calculate the distance between real vectors using the sum of their absolute difference. Also called City Block Distance.
- **Minkowski Distance** — Generalization of Euclidean and Manhattan distance.
- **Cosine Distance** — This distance metric is used mainly to calculate similarity between two vectors. It is measured by the cosine of the angle between two vectors and determines whether two vectors are pointing in the same direction. It is often used to measure document similarity in text analysis. When used with KNN this distance gives us a new perspective to a business problem and lets us find some hidden information in the data which we didn't see using the above two distance matrices.
- **Jaccard Distance** — The Jaccard coefficient is a similar method of comparison to the Cosine Similarity due to how both methods compare one type of attribute distributed among all data. The Jaccard approach looks at the two data sets and finds the incident where both values are equal to 1. So the resulting value reflects how many 1 to 1 matches occur in comparison to the total number of data points. This is also known as the frequency that 1 to 1 match, which is what the Cosine Similarity looks for, how frequent a certain attribute occurs.
- Etc.

Steps to be carried out during the K-NN algorithm are as follows:

1. Divide the data into training and test data.
2. Select a value K.
3. Determine which distance function is to be used.
4. Choose a sample from the test data that needs to be classified and compute the distance to its n training samples.
5. Sort the distances obtained and take the k-nearest data samples.
6. Assign the test class to the class based on the majority vote of its k neighbors.



2 — Performances and characteristics of K-Nearest Neighbors Classification Algorithm

K-Nearest Neighbor (KNN) Algorithm

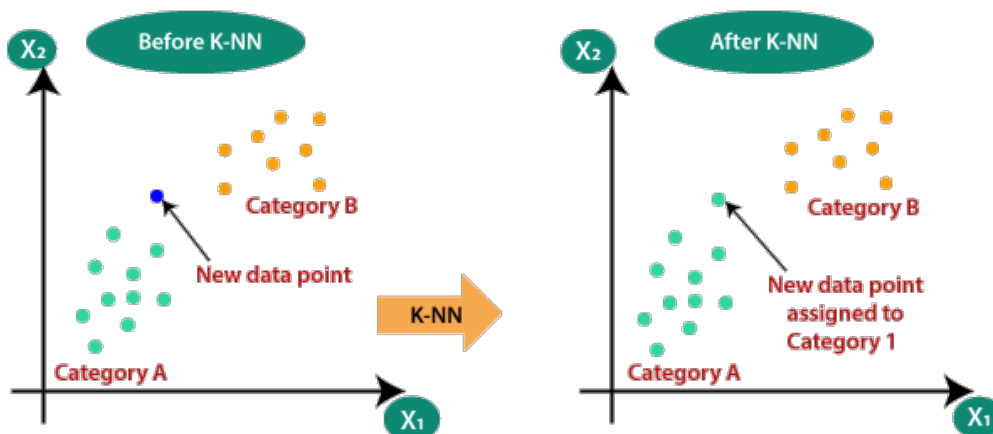
- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



Why do we need a K-NN Algorithm?

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



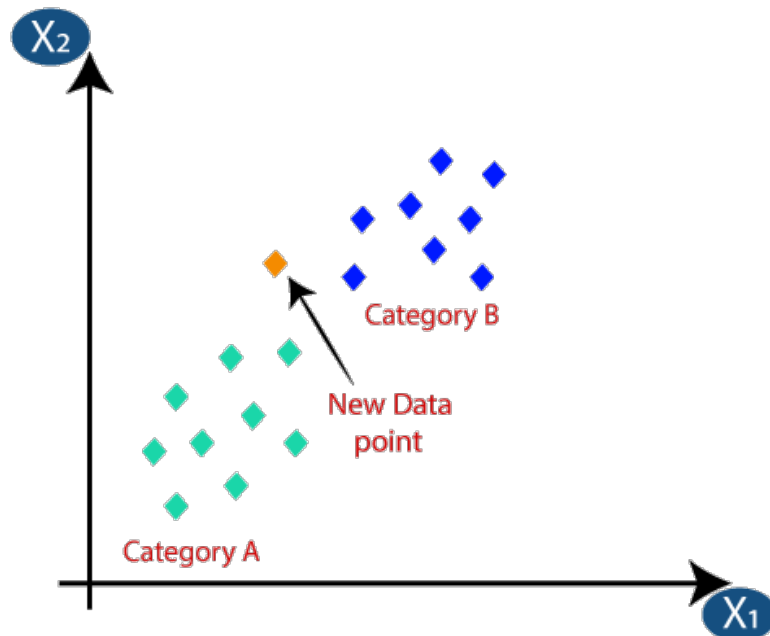
How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

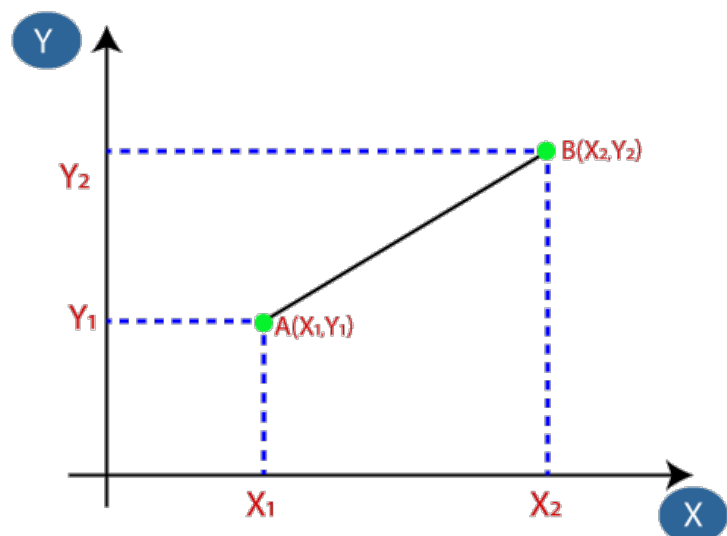
- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.

- **Step-6:** Our model is ready.

Suppose we have a new data point and we need to put it in the required category. Consider the below image:



- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

How to choose K?

- Deciding the k can be the most critical part of K-nearest Neighbors.
- If the value of k is small then noise will have a higher dependency on the result. Overfitting of the model is very high in such cases.
- The bigger the value of K will destroy the principle behind KNN.
- You can find the optimal value of K using *cross-validation*.
- Value of k – bigger the value of k increases confidence in the prediction.
- Decisions may be skewed if k has a very large value.

Performance of the K-NN algorithm is influenced by three main factors:

1. The **distance function** or distance metric used to determine the nearest neighbors.
2. The **decision rule used to derive a classification** from the K-nearest neighbors.
3. The **number of neighbors** used to classify the new example.

Advantages of K-NN :

1. The K-NN algorithm is very easy to implement.
2. Nearly optimal in the large sample limit.
3. Uses local information, which can yield highly adaptive behavior.
4. Lends itself very easily to parallel implementation.
5. It is simple to implement.
6. It is robust to the noisy training data
7. It can be more effective if the training data is large.
8. A simple algorithm that is easy to understand.
9. Used for nonlinear data.
10. The versatile algorithm used for both classification as well as regression.
11. Gives high accuracy but there are more good algorithms in supervised models.
12. The algorithm doesn't demand to build a model, tune several model parameters, or make additional assumptions.

Disadvantages of K-NN:

1. Large storage requirements.
2. Computationally intensive recall.
3. Highly susceptible to the curse of dimensionality.
4. Always needs to determine the value of K which may be complex some time.
5. The computation cost is high because of calculating the distance between the data points for all the training samples.
6. The requirement of high storage.
7. Prediction rate slow.
8. Stores all the training data.
9. The algorithm get slower when the number of examples, predictors or independent variables increases.
10. The KNN is out perform when the data is very **noisy** or is not **linearly separable**.
11. KNN, at times, can be costly in terms of **computations** for larger datasets.
12. KNN is least preferred when datasets have **multiple dimensions**, as it leads to **a curse of dimensionality**.

K-NN Algorithm finds its applications in:

1. Finance — financial institutes will predict the credit rating of customers.
 2. Healthcare — gene expression.
 3. Political Science — classifying potential voters in two classes will vote or won't vote.
 4. Handwriting detection.
 5. Image Recognition.
 6. Video Recognition.
 7. Pattern Recognition.
-